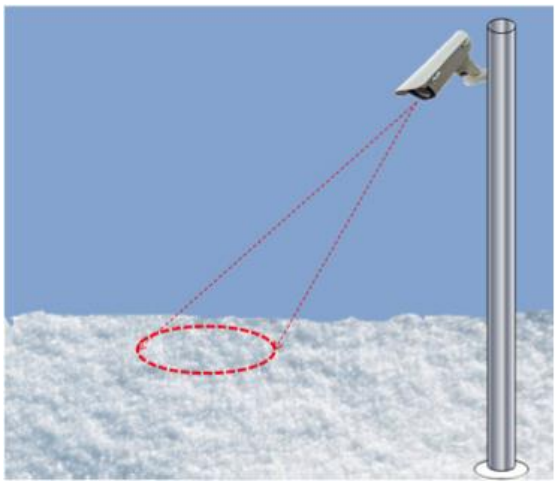
Dokumentation

Schneemessapplikation SW



Autor:	Mauro Blatter
Erstelldatum:	01.01.2023
Abgabedatum:	02.04.2023
Lehrer:	Felix Räschle
Schule:	Gewerbliche Berufsschule
Lehrmeister:	Chris Schmidt
Firma:	CEDES AG

Inhaltsverzeichnis

1 Einführung 4

2 Geplantes Projekt 4
2.1 Anforderungen an das Projekt 4
2.2 Ziele des Projektes 4
2.3 Projektbeteiligungen 5
2.4 Kosten 5

3 Begriffserklärungen 6
3.1 Was ist LoRa? 6
3.2 Was ist LIDAR? 7

4 Konzepte 8
4.1 Hardwareaufbau Sensor 8
4.2 Softwarekonzept 9

5 Software 11
5.1 Software «Host» 12
5.1.1 Mapping 12
5.1.2 Request Handler 15
5.1.3 Lora Handler 18
5.1.4 Timer 20
5.2 Software «Sensor» 21
5.2.1 Bme680 21
5.2.2 LidarSensor 23
5.2.3 Lora 25
5.2.4 Main 26

6 Fehler 27

8 Projektzukunft 28

9 Schlusswort 28

9.1 Fazit 28

9.2 Danksagung ..... **Fehler! Textmarke nicht definiert.**

10 Berechnungen 28

11 Mitliegende Unterlagen 28

12 Abbildungsverzeichnis 29

13 Quellenverzeichnis 30

1 Einführung

Zu Beginn des Schuljahres, im August 2021 wurde das Schulfach „Bereichsübergreifende Projekte“ eingeführt. Für dieses Fach müssen alle Lernenden ein Projekt erstellen, welches grundsätzlich mit dem Beruf Elektroniker EFZ zu tun hat. Im ersten Jahr habe ich die Hardware und die Firmware für eine Tastatur fertiggestellt.

Der späteste Abgabetermin der gesamten Projektunterlagen ist am 2. April 2023.

Das persönliche Ziel meinerseits ist es, dass ich mich mit der Planung sowie dem Erstellen von Firmware weiterentwickeln kann, und viel Neues lernen kann. Ich möchte ebenfalls, dass mich diese Erfahrung für weitere Projekte weiterbringt.

2 Geplantes Projekt

Ich hatte lange Zeit keine Ahnung was für ein Projekt ich eigentlich machen könnte. Erst überlegte ich mir mein vorheriges Projekt zu verbessern und anzupassen. Doch dann sagte ein Klassenkollege, dass ich als Engadiner ein Schneehöhenmessgerät machen könnte, da dort im Normalfall viel Schnee liegt. Zu Beginn machte ich noch Witze darüber. Doch erst nach längerem überlegen erkannte ich das Potential eines solchen Projektes und begann sofort mit der Ausarbeitung der Spezifikation. Die gesamte Schneemessapplikation soll vollumfänglich funktionieren. Sie soll aus zwei Teilen bestehen, dem Sensor «Slave» und dem Pi - Server «Master». Als User soll man von zuhause aus mit dem Browser Anfragen an den «Master» senden können, welcher dann die Entsprechende Information beim Sensor anfordert. Dieser soll sie dann zurück an den Master senden, welche sie Grafisch darstellt. Der User soll zudem in der Lage sein Referenzmessungen für einen neuen «Referenzpunkt» anzufordern. Das Ganze soll mit dem http Protokoll realisiert werden.

2.1 Anforderungen an das Projekt

Alle vorgegebenen Zeiten sowie alle weiteren Vorgaben sollen eingehalten werden. Alle Projektdokumente sollten vollständig und übersichtlich sein.

2.2 Ziele des Projektes

- **Dokumentation:**
  Die Dokumentation wird nach Vorgabe mit Microsoft Word 2016 erstellt. Zudem wird sie fortlaufend ergänzt und soll am 22. März 2023 mit allen anderen Dokumenten abgegeben werden.

- **Software Schreiben und test**
  Der Master Softwareteil wird in der Sprache «Python» geschrieben, der Slave Teil mit «Circuitpython». Dazu verwendet wurde Visual Studio Code 23 mit den jeweiligen Python und Circuitpython Paketen. Der Code soll laufend getestet werden.

- **Schneehöhenmessung**
  Die Schneehöhenmessung soll nur dann ausgeführt werden, wenn eine vom Master (User) angefordert wird. Diese soll gemeinsam mit der Temperatur und der Höhe (m.ü.m) dargestellt werden. Die Referenzmessung (zu welchem Wert der Unterschied gemessen wurde) soll auch dem User zur Verfügung gestellt werden.

- **Übertragung**
  Der Master soll nach erhaltener Anfrage die Daten über LORA-Funk and den Slave senden, welche sie ausliest, anhand von diesen ausführt und die Messdaten zurücksendet.

- **Schnittstelle**
  Das http Mapping soll über die Anfragen «GET», «POST» und «DELETE» verfügen.
  - GET soll einen Wert bzw. mehrere Werte in einem bestimmten Zyklus messen.
  - POST soll einen neuen Referenzwert setzten.
  - DELETE soll eine die letzte Referenzmessung löschen.

2.3 Projektbeteiligungen

Ziel ist, dass der Lernende das Projekt selbstständig anfertigt. Falls er jedoch Hilfe benötigt, kann er diese bei Lehrer, Lehrmeister oder anderen Firmenangestellten einholen.

2.4 Kosten

Jegliche Bauteilbeschaffungskosten wurden im Wert von ca. 197.00 CHF von CEDES AG übernommen.

3 Begriffserklärungen

Zum besseren Verständnis einzelner Begriffe sollen diese erklärt werden.

3.1 Was ist LoRa?

LoRa bedeutet so viel wie «**Lo**ng **Ra**nge». Es ist eine Funktechnik die nicht nur sehr stromsparend funktioniert jedoch auch für lange Distanzen geeignet ist. Die Einzige Abstriche gibt es bei der Datenübertragungsrate. Die Reichweiten sind in der Regel über 10 Kilometer. Jedoch kann diese in starkbesiedelten Gebieten stark abweichen. Ein weiterer Vorteil ist, dass Gebäude durchdringt, werden können und so auch Kellerabteile erreicht werden können.

Die Übertragungsfrequenzen von Lora betragen im Europäischen Raum zwischen 433 – 434 Mhz, und von 863 – 870 Mhz. In anderen Kontinenten sind diese jedoch unterschiedlich.
Da die verwendeten Frequenzen öffentlich sind, sollten Lora Geräte grundsätzlich so konzipiert sein, dass die Kommunikation nicht im Sekundentakt funktioniert, sondern nur im grösseren Minutentakt damit die Frequenzen nicht überlastet werden.

Abbildung 1: Lora Übersicht

3.2 Was ist LIDAR?

LIDAR bedeutet **L**ight **D**etection **A**nd **R**anging und wird normalerweise zur Umfelderfassung benutzt. So wird ein gepulster Lichtlaser eingesetzt um Objekte zu erkennen. LIDAR Technologien werden normalerweise in der Atomphysik, Archäologie, Robotik oder in der Automobilindustrie verwendet.

Lidar Sensoren senden gepulste Lichtwellen aus, diese werden dann von einem potenziellen Objekt reflektiert und kehren zum Sensor zurück. Dadurch kann der Sensor die Distanz bis zum Objekt berechnen. Dieser Vorgang ist ähnlich wie beim ToF verfahren. Zudem wiederholt dies LIDAR für mehrere Punkte, wodurch eine gesamte Fläche „angeschaut“ wird, und dadurch ein 3D Objekt erstellt werden kann. So können nicht nur genauere, sondern auch zuverlässige Ergebnisse bei schlechtem Wetter oder Schnee erzielt werden. Daher ist der Einsatz von LIDAR Sensoren heutzutage im Bereich der Schneehöhenmessung von Pistenmaschinen üblich.

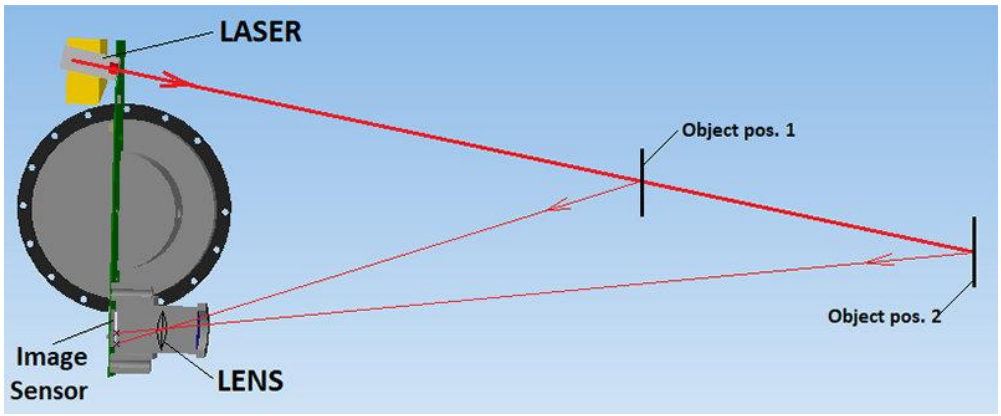


Abbildung 2: LIDAR Funktion

4 Konzepte

Das Blockschaltbild der Hardware ist klein und einfach. Dies ist auch der Grund, dass kein Schema für das Projekt angefertigt wurde. Zudem liegt der Fokus des Projektes auf der Software und nicht auf Hardware, dies ist auch der Grund warum nicht genauer auf die Komponentenwahl und Spezifikationen eingegangen wird.

4.1 Hardwareaufbau Sensor

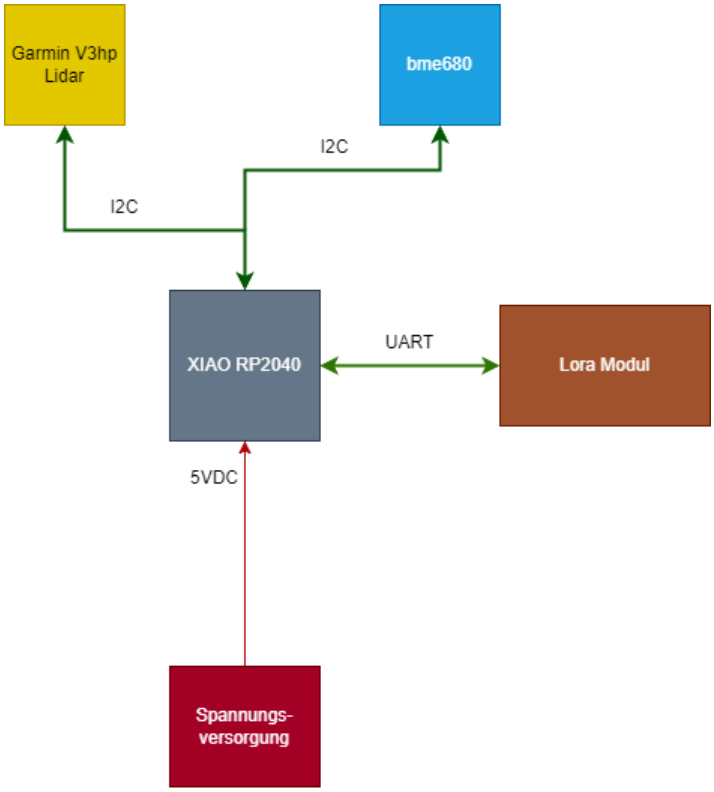


Abbildung 3: HW Übersicht

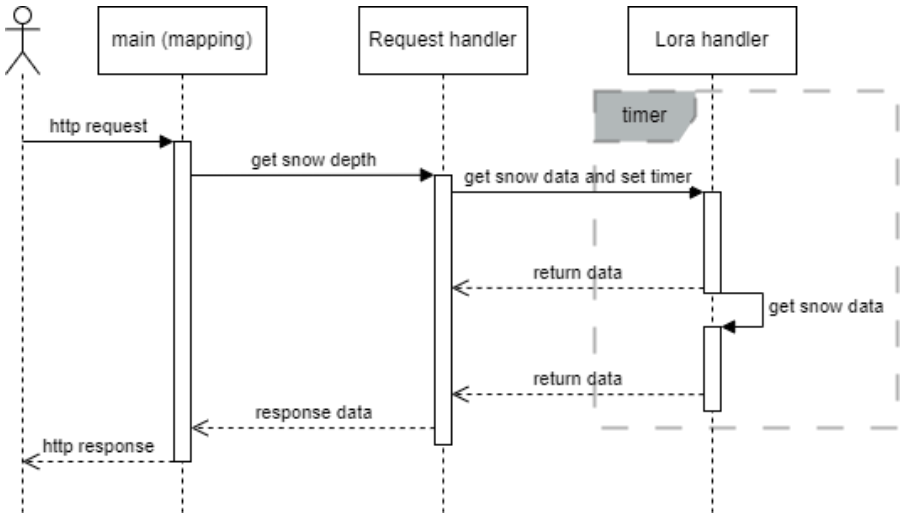
Sensor: Die Spannungsversorgung liefert dem Mikrocontroller (XIAO RP2040) 5VDC. Von der gleichen Spannung wird auch der bme680 und der Lidar Sensor gespiesen. Dieser ist jedoch zusätzlich noch über ein 680uF Kondensator verbunden. Das Lora Modul erhält die Spannung (3.3VDC) direkt vom Mikrocontroller. Wichtig zu beachten war, dass die RX/TX Pins (UART) vertauscht angeschlossen werden.

Beim «Host» wird ein RockPi4B mit 5VDC gespiesen. Das Lora Modul wird dort ebenfalls mit vertauschten RX/TX Pins und einer 3.3VDC Spannung direkt verbunden.

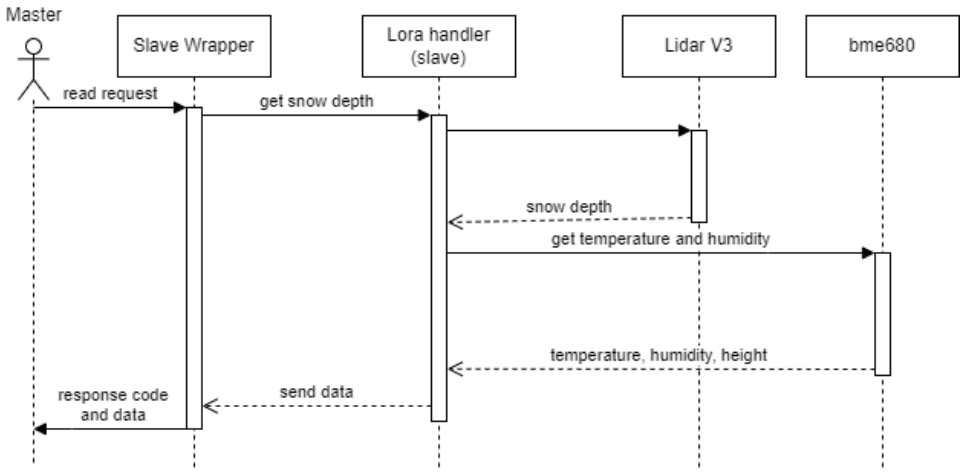
4.2 Softwarekonzept

Der Code soll in zwei Teile aufgeteilt sein (Master und Slave):

Master:
Jederzeit sollen http Anfragen entgegengenommen werden. Mit dem Python Webframework Flask soll dies sichergestellt werden. Sobald eine Anfrage kommt, wird diese anhand vom Command eingeordnet.
Im Fall einer optionalen Messrepetitionsangabe sollen die Daten über den Header mitgegeben werden. Der Entsprechende Timer soll ein eigenes Modul erhalten. Aufgerufen und gestartet soll er im Request Handler werden. Der Timer soll alle «n» Stundenzahl eine Anfrage an den Slave senden. Der Slave sendet nach auswerten der Anfrage die Daten über Lora zurück zum «Host». Nach empfangen der daten werden diese ausgelesen und ausgewertet. Sie werden direkt zum «main» Modul weitergeleitet, wodurch sie mit Flask für den Benutzer zugänglich gemacht werden. Im Falle eines Fehlers wird eine entsprechende Nachricht sowie der dazugehörige http-response code angezeigt.
Im Falle einer Delete Anfrage wird diese Anfrage an den Sensor (dem Slave) über Lora gesendet und lokal wird die letzte Referenzmessung gelöscht. Der Wert der Referenz wird auf 0 zurückgesetzt.
Nur mit einer Post Anfrage kann erneut eine neue Referenzmessung gesetzt werden. Wenn noch keine Referenzmessung vorhanden ist, wird der User eine Negativzahl erhalten.
Der Ablauf einer Anfrage erfolgt immer über das gleiche oben beschriebene Muster.



Slave:
Sobald eine Datenanfrage von Lora über UART kommt, wird diese verarbeitet. Vor jeder Messausführung wird die Luftfeuchtigkeit gemessen, und es wird nur die Operation ausgeführt, wenn der Wert der Luftfeuchtigkeitsmessung unter einem bestimmten Wert ist. Entsprechend wird diese Anfrage dem Lidar Sensor Modul zugewiesen, welcher entweder eine neue Referenz oder normale Messung durchführt oder den Referenzwert mit 0 ersetzet. Der Wert davon wird mit einem bestimmten Wert verglichen. Sofern die Luftfeuchtigkeit nicht zu hoch ist, wird im Lidar Sensor Modul die Distanz gemessen. Dies soll ungefähr 99–100-mal ausgeführt werden, und danach den Durchschnitt daraus gezogen werden. Danach wird die Anfrage an das bme680 Modul weitergeleitet, worüber die Temperatur und Höhenmessung erfolgen. Alle gemessenen Daten werden dann im Lora Modul als Antwort gesendet. Falls eine der Messungen nicht ausgeführt werden kann, wird eine Fehlerzahl z.b 404 gesendet. Im Falle, dass die Luftfeuchtigkeit zu hoch ist, wird eine andere Fehlerzahl gesendet.
Der Controller kommuniziert mit dem Lidar sowie bme680 Sensor über I2C.



5 Software

Die gesamte Software ist übersichtshalber über mehrere Files verteilt. Das main.py File, ist jenes worin das Mapping des Flask Servers läuft. Grundsätzlich wurde die Empfehlung beachtet, pro Modul eine Klasse zu haben.

5.1 Software «Host»

5.1.1 Mapping

```
8   @SnowDepthApp.route("/snowbla/snowdata/<rep>", methods=['GET'])
9   def snow_depth(rep):                                                    1
10      try:
11          data = req_handler.get_timed_snow_depth(rep)
12          return data, HTTPStatus.OK
13
14      except Exception:
15          return "error occured during measurement", HTTPStatus.NO_CONTENT
16
17  @SnowDepthApp.route("/snowbla/reference/", methods=['GET'])
18  def get_reference():                                                    2
19      try:
20          ref = req_handler.get_reference()
21          return ref, HTTPStatus.OK
22
23      except Exception:
24          return "not able ro read reference", HTTPStatus.NO_CONTENT
25
26
27  @SnowDepthApp.route("/snowbla/reference/", methods=['POST'])
28  def new_reference():                                                    3
29      try:
30          new_ref = req_handler.new_reference()
31          return new_ref, HTTPStatus.OK
32
33      except Exception:
34          return "not able to create a new reference", HTTPStatus.NO_CONTENT
35
36
37  @SnowDepthApp.route("/snowbla/reference/", methods=['DELETE'])
38  def delete_reference():                                                 4
39      try:
40          req_handler.delete_reference()
41          return "reference deleted successfully", HTTPStatus.OK
42
43      except Exception:
44          return "unable to locate and delete reference", HTTPStatus.NO_CONTENT
45
```

Abbildung 4: main.py

SnowDepthApp wird zuvor als Flask App Objekt festgelegt. Ausserdem wird zu Beginn ein Objekt aus der Klasse CRequestHandler erstellt. Die Klasse CRequestHandler befindet sich im Modul request_handler.py.

```
SnowDepthApp = Flask(__name__)
req_handler = CRequestHandler()
```

Abbildung 5: Objekt Flask

1. Funktion und mapping für snow_depth (schneehöhe). Das eigentliche Mapping läuft über /snowbla/snowdata/<wann eine messung>, dieses «wann eine Messung» bzw «rep» im code, stellt eine Integer zahl dar, all wie viele Stunden eine Messung gemacht werden sollte, und diese daten zurückgegeben werden sollten. Zudem sind nur die http Methoden «GET»

   erlaubt. Dies wird mit «methods=[GET]» sichergestellt, und wird so vom Flask Framework definiert.

   Danach wird versucht die Funktion «get_timed_snow_depth» mit der Stundenanzahl auszuführen. Sofern dies funktioniert, werden die Daten sowie ein http Response Code «OK» an den User zurückgegeben.

   Falls jedoch ein Fehler aufkommt, wird dieser durch das «exception» aufgefangen, und es wird ein http Response Code «No Content» zurückgegeben. Zusätzlich gibt es noch eine für den User lesbare Nachricht.

2. Funktion und mapping für get_reference (gebe referenz). Das Mapping läuft über /snowbla/reference/. Hierbei kann kein Parameter definiert oder mitgegeben werden. Zudem sind nur die Methoden «GET» erlaubt
   So wird versucht die Funktion «get_reference» im Modul request_handler.py auszuführen. Falls dies funktioniert, wird dem User die Referenz Messung «ref» zur Verfügung gestellt. Ebenfalls kommt ein http Response Code «OK» zurück.
   Falls jedoch ein Fehler aufkommt, wird eine Fehlernachricht sowie der http Response Code «No Content» dem User zurückgegeben.

3. Funktion new_reference (neue referenz). Das Mapping funktioniert über /snowbla/reference/. Ebenfalls kann kein Wert oder Parameter mitgegeben werden. Erlaubt ist hierbei nur die Methode «POST».
   Es wird versucht die Funktion «new_reference» aus dem request_handler.py Modul auszuführen. Falls dies funktioniert, wird der Wert der neuen Referenzmessung sowie der http Response Code «OK» dem Benutzer angezeigt.
   Im Fehlerfall wird nur eine Fehlernachricht und der http Response Code «No Content» dem User zur Verfügung gestellt.

4. Funktion delete_reference (lösche referenz). Das Mapping läuft ebenfalls über «/snowbla/reference/». Auch hier kann kein Wert mitgegeben werden. Die erlaubten http Methoden sind hierbei «DELETE». So wird versucht die Methode «delete_reference» aus dem request_handler Modul auszuführen. Falls dies gelingt, wird eine Nachricht sowie ein http Code «OK» zurückgegeben.
   Falls jedoch ein Fehler aufkommen sollte, wird eine Fehlernachricht sowie ein Response Code «No Content» dem Benutzer zurückgegeben.

Damit der Server bzw. der Host gestartet werden kann, ist noch folgendes nötig:

```
if __name__ == '__main__':
    SnowDepthApp.run(host="0.0.0.0", port="5000", debug=True)
```

Abbildung 6: server run

Mit SnowDepthApp.run wird das Flask objekt gestartet. Das «.run» kommt so vom Flask Webframework. Mitgegeben werden müssen noch die IP und der Port.
Die IP «0.0.0.0» bedeutet, dass der Server auf allen Netzwerkschnittstellen des Computers (RockPIs) verfügbar sein wird.
Der definierte Port «5000» ist neben «8080» ein Standardport, jedoch ist dieser frei wählbar.

5.1.2 Request Handler

```
class CRequestHandler:

    def __init__(self):
        self.Lora = CLoraHandler(0x50)
        self.timer = None
```

Abbildung 7: request handler

Im Konstruktor wird das Objekt Lora definiert. Zusätzlich wird ein Klassenattribut «timer» mit dem Wert None initialisiert.

```
def get_timed_snow_depth(self, time_cycle=0):
    """receives the snow depth data

    Args:
        time_cycle: int only, repetition interval in seconds, default = 0

    Returns:
        data: snow depth from sensor (float)
    """
    self.Lora.data = 0x01
    if self.timer is not None:
        self.timer.stop()
    self.timer = CTimer(int(time_cycle), self.Lora.send_data)
    self.timer.start()
```

Abbildung 8: Funktion get_timed_snow_depth

Die Variabel «time_cycle» widerspiegelt die Anzahl bzw. den Interval der angeforderten Messungen.

Das der Lora Klasse zugehörige Attribut «data» wird auf 0x01 gesetzt. Danach wird geprüft ob ein laufender Timer vorhanden ist, wenn einer vorhanden ist wird dieser gelöscht, da ein neuer gesetzt werden wird. «timer» wäre im Falle dass bereits einer vorhanden ist, ein Objekt und somit nicht None.
Danach wird mit dem Interval ein neuer Timer gestartet. Ebenfalls wird dem Timer gesagt welche Funktion er alle «n (time_cycle)» Sekunden ausführen soll. Danach werden noch die Daten von Lora empfangen und zurückgegeben.

```python
def new_reference(self):
    """create a new reference

    Returns:
        data: new reference (float)
    """
    if self.timer is not None:
        self.timer.stop()

    self.Lora.data = 0x10
    self.Lora.send_data()
    data = self.__receive_data()
    return data
```

Abbildung 9: Funktion new_reference

Auch hier wird geschaut ob ein aktiver Timer vorhanden ist, falls dies zutrifft, wird er gestoppt.

Das dem Lora Objekt angehörige Attribut data wird der Wert 0x10 übergeben.

Im Anschluss werden die Daten über die Funktion «send_data» gesendet. Nach empfangen der Daten werden sie zum Flask zurückgegeben.

```python
def get_reference(self):
    """get the current reference

    Returns:
        ref: reference data (float)
    """
    if self.timer is not None:
        self.timer.stop()

    self.Lora.data = 0x11
    self.Lora.send_data()
    ref = self.__receive_data()
    return ref
```

Abbildung 10: Funktion get_reference

Die Funktion «get_reference» macht genau das gleiche, nur dass die Lora Daten 0x11 übergeben werden.

```python
def delete_reference(self):
    """delete the current reference (set to 0)

    Returns:
        old_ref: data of old_ref which no longer exists
    """
    if self.timer is not None:
        self.timer.stop()

    self.Lora.data = 0x12
    self.Lora.send_data()
    old_ref = self.__receive_data()
    return old_ref
```

Abbildung 11: Funktion delete_reference

Gleich sieht es auch für die Funktion zum Löschen der Referenz aus. Hier werden die Daten auf 0x12 gesetzt.

```python
def __receive_data(self):
    """receive data from lora handler

    Returns:
        data: received data from lora handler
    """
    data = self.Lora.receive_data()
    return data
```

Abbildung 12: Private Funktion receive_data

Die Private Funktion «receive_data» macht nichts anderes, als dass sie die Funktion «receive_data» des Lora Objektes ausführt, und die erhaltenen Daten zurückgibt.

5.1.3 Lora Handler

```
# serial bus (uart2): /dev/ttyS2
class CLoraHandler:

    def __init__(self, data):
        self.bus = Serial("/dev/ttyS2", baudrate=9600, parity="none", stopbits=1, databits=8)
        self.data = data
```

Abbildung 13: LoraHandler

Der Konstruktor von der Klasse LoraHandler besitzt die beiden Attribute bus und data. Das Attribut bus ist ein Objekt von der Klasse Serial. Dies wird für die Funktion des Lora (UART) bus benötigt. Mitgegeben wird der Bus («ttyS2»), die Baudrate sowie weitere UART-Einstellungen welche so Standard sind.

Das Attribut data muss beim Erstellen eines Objektes übergeben werden. Dieses entspricht den Daten welche Lora sendet.

```
def send_data(self):
    """lora send data

    Returns:
        TypeError: when invalid datatype tried to send
        ValueError: when invalid data in bytes
    """
    try:
        length = self.bus.write([self.data])
        # return length

    except TypeError:
        return "datatype is inavlid"

    except ValueError:
        return "data invalid in bytes"
```

Abbildung 14: Funktion send_data

Die Funktion «send_data» sendet die Daten welche als «data» Attribut gesetzt ist. Zurückgegeben wird bei korrekter Funktion nichts. Nur im Fehlerfall wird eine Rückgabe gemacht. Diese spielt jedoch keine Rolle und dient mehr der Verständlichkeit, da die Exception im main Modul aufgefangen wird, und dem User keine genauen Fehlercodes zur Verfügung gestellt werden.

```python
def receive_data(self):
    """lora receive data

    Returns:
        data: received data
        Error: if error during reading data (uart lora)
    """
    try:
        data = self.bus.read(2)
        self.data = CDataElements(data[1], data[2])
        return data

    except Exception:
        return "bad request, no data recevied to to any error"
```

Abbildung 15: Funktion receive_data

Empfangen und gelesen werden zwei Bytes. Die empfangenen Bytes werden auch der Datenklasse DataElements zur Verfügung gestellt. Diese soll später dafür genutzt werden, um multiple Datensätze zu speichern, um sie dann grafisch darzustellen.

Zurückgegeben werden die Daten nur sofern keine Fehler während dem Empfangen aufkommen. Ansonsten wird auch hier ein Fehler zurückgegeben, welcher dann von main aufgefangen wird.

5.1.4 Timer

```
class CTimer():

    def __init__(self,reps,do_function):
        self.reps=reps
        self.do_function = do_function
        self.timed_thread = Timer(self.reps,self.handle_function)

    def handle_function(self):
        """new thread for each timer operation"""
        self.do_function()
        self.timed_thread = Timer(self.reps,self.handle_function)
        self.timed_thread.start()

    def start(self):
        """start timer"""
        self.timed_thread.start()

    def stop(self):
        """stop timer"""
        self.timed_thread.cancel()
```

Abbildung 16: Timer

Dem Timer werden die Variabeln reps und do_function übergeben. Das attribut «do_function» entspricht der Funktion, welcher der Timer in einem gewissen Intervall ausführen soll. Auch wird Im Konstruktor ein thread mit dem Objekt Timer von «threading» erstellt.

Die Funktion «handle_function» wird dafür benutzt, damit ein neuer Thread benutzt wird.

Weiter gibt es noch die Funktionen start und stop, welche nur benutzt werden um den Timer zu starten bzw. zu stoppen.

Die gesamte Timer Funktion wurde so von einem Beispiel übernommen. Ich habe mich nicht vertieft mit dem Thema «Threads» oder «Timer» auseinandergesetzt.

5.2 Software «Sensor»

5.2.1 Bme680

```
class Cbme680:
    def __init__(self, sea_level_pressure=1000):
        self.bme680i2c = busio.I2C(board.SCL, board.SDA)
        self.bme680 = adafruit_bme680.Adafruit_BME680_I2C(self.bme680i2c, address=0x76)
        self.bme680.sea_level_pressure = sea_level_pressure
```

Abbildung 17: Klasse bme680

Im Konstruktor der Klasse bme680 wird das I2C Interface (die Pins) der Klasse adafruit_bme680 (bme680 lib) bei der Objektinstanzierung übergeben. Dort wird auch die Adresse 0x76 definiert. Zudem wird die Klassenvariabel «sea_level_pressure» mit dem Default Wert 1000 initialisiert. Dieser Wert ist jedoch anpassbar.

```
def get_temperature(self):
    """method used to measure temperature. This is done with 10 measurements

    Returns:
        float: mean of 10 single temperature measurements in degree Celsius
    """
    temperature_list = []
    for x in range(10):
        temp = self.bme680.temperature
    temperature_list.append(temp)

    mean_temp = sum(temperature_list)/len(temperature_list)
    return float(mean_temp)
```

Abbildung 18: Funktion get_temperature

```python
def get_altitude(self):
    """method used to measure altitude of sensor. Be aware to set according sea level pressure

    Returns:
        float: altitude in meters over sea level
    """
    return float(self.bme680.altitude)
```

Abbildung 19: Funktion get_altitude

Die Funktion get_temperature wird dafür benutzt, um die Temperatur auszulesen. Damit ein genaueres Ergebnis gemessen wird, wird die Messung 10 mal Wiederholt. Anschliessend wird daraus der Durchschnitt berechnet. Dies ist besonders hilfreich, damit eine einzelne Falsche Temperaturmessung «Ausreisser» keinen groben Einfluss hat. Zurückgegeben wird eine Kommazahl (float).

Die Funktion get_altitude gibt eine Floatzahl zurück welche der ungefähren Sensorhöhe (über Meer) entspricht. Für diese Messung braucht die bme680 Adafruit Library die Variabel «sea_level_pressure». Jeh nach Höhe des Sensors (Standorts) muss diese entsprechend angepasst werden. Nach mehreren Tests habe ich diese auf 1000, bei einer Höhe von ca. 1750 m.ü.m festgelegt. Von Adafruit vorgeschlagen sind 1013.

```python
def get_humidity(self):
    """method used to measure humidity

    Returns:
        float: humidity in percentage
    """
    return float(self.bme680.humidity)
```

Abbildung 20: Funktion get_humidity

Damit die Luftfeuchtigkeit ausgelesen werden kann, wird dies mit der Funktion get_humidity gemacht. Diese gibt eine Kommazahl zurück in Prozent (float).

```python
class CLidarSensor:
    def __init__(self, measure_repetitions = 99):
        self.lidari2c = busio.I2C(board.SCL, board.SDA)
        self.sensor = adafruit_lidarlite.LIDARLite(self.lidari2c, sensor_type=adafruit_lidarlite.TYPE_V3HP)
        self.repetitions = measure_repetitions
```

Abbildung 21: Klasse LidarSensor

5.2.2 LidarSensor

Im Konstruktor der Klasse LidarSensor wird das Attribut Sensor mit den I2C Pins sowie dem Sensortyp mit der Klasse adafruit_lidarlite instanziert. Diese Klasse ist aus der lidarlite library von adafruit. Zudem wird die Variabel measure_repetitions mit dem Default wert 99 als Klassenattribut initialisiert.

Mit der Funktion get_distance wird die Distanz gemessen. Die eigentliche

```python
def get_distance(self):
    """method used to measure distance. Default repetition value at 99, due to bias measurement at every 100th.

    Returns:
        float: mean of measure repetition values in cm
        float: bias distance in case default value of repetiotions at 99 is used in cm
    """
    distance_list = []
    for x in range(self.repetitions):
        distance = self.sensor.read_distance_v3hp()
    distance_list.append(distance)

    if self.repetitions == 99:
        bias_dist = self.sensor.read_distance_v3hp()
    else:
        bias_dist = None

    mean_dist = sum(distance_list)/len(distance_list)
    return float(mean_dist), float(bias_dist)
```

Abbildung 22: Funktion: get_distance

Distanzmessung wird «n (repetitions)» ausgeführt, und das Resultat in eine Liste geschrieben. Gleichzeitig wird auch eine Bias Messung ausgeführt (falls der repetitions Wert 99 entspricht). Diese Messung ist eine genauere Messung als die anderen, sie benötigt auch mehr Strom. Danach wird der Durchschnitt aus der zuvor geformten Liste berechnet. Die erhaltene Zahl entspricht der Distanz, welche auch gemeinsam mit dem Bias Messwert zurückgegeben wird. In Tests hat sich gezeigt, dass das Resultat im Schnitt von 100 bzw. 99 Messungen am genausten ist, um einiges genauer als mit z.b 10. Der Unterschied zum Resultat mit 200, oder 1000 Messungen ist jedoch nur minim.

```python
def get_bias_distance(self):
    """method used to do a bias measurement while ignoring all other

    Returns:
        float: bias distance value in cm
    """

    for x in range(99):
        self.sensor.read_distance_v3hp()
    bias_distance = self.sensor.read_distance_v3hp()

    return float(bias_distance)
```

Abbildung 23: Funktion get_bias_distance

Die Funktion get_bias_distance wird dafür genutzt, um eine Bias Messung durchzuführen. Der Sensor macht automatisch bei jeder 100. Messung eine Bias Messung, daher wird 99. Mal gemessen, und nur die letzte zurückgegeben. (float)

Zusätzlich wurde noch eine weitere Funktion geschrieben, um manuell einen Wert ins Register zu schreiben. Diese wird jedoch nicht im Projekt verwendet, sondern dient für das Zukünftige fortführen des Projektes.

```python
def set_configuration(self, configuration_type: int):
    """method used to change sensor configuration type

    Args:
        configuration_type (int): https://docs.circuitpython.org/projects/lidarlite/en/latest/api.html

    default value if no or invalid argument is passed
    """
    if configuration_type < 10:
        self.sensor.configure(config=configuration_type)
    else:
        self.sensor.configure(config=0)
```

Abbildung 24: Unbenutzte Funktion set_configuration

5.2.3 Lora

```
class CLora():
    """Class CLora used to handle (receive and send) data from lora transmission"""
    def __init__(self):
        self.uart = busio.UART(board.TX, board.RX, baudrate=9600, bits=8, parity=None, stop=1)
```

Abbildung 25: Klasse Lora

Das Klassenattribut uart welche mit der Klasse UART instanziiert wird, stellt den Uart Bus dar. Dafür nötig sind die RX/TX Pins, Baudrate und noch weitere Parameter.

```
def lora_receive(self):
    """Method to receive data by lora

    Returns:
        str: sent data
        bool: False, if data is none
    """
    data = self.uart.read(1)
    if data is not None:
        return data
    return False
```

Abbildung 26: Funktion lora_receive

Die Funktion lora_receive wird benutzt, um Lora daten zu empfangen. Die Zahl in uart.read, stellt die Anzahl der zu lesenden Bytes dar. Da ich höchstens 1 Byte sende, reicht dieses aus. Zurückgegeben werden die daten, sofern diese nicht None sind.

Die Funktion lora_send wird benutzt, um Daten über Lora zu senden. Falls kein Fehler währenddessen aufkommt, wird ein True Wert zurückgegeben, ansonsten False.

```
def lora_send(self, data: bytes):
    """Method to send data over lora

    Returns:
        bool: True if data sent
    """
    try:
        self.uart.write(bytes([data]))
        return True
    except Exception:
        return False
```

Abbildung 27: Funktion lora_send

5.2.4 Main

```
MEASURE_ACCURACY = 100

lora = CLora()
lidar = CLidarSensor(MEASURE_ACCURACY)
bme680 = Cbme680()

reference = 0

while True:
    data = lora.lora_receive()
    time.sleep(0.01)
    if data is not False:
        hum = bme680.get_humidity()
        if hum < 65.0:
            data = hex(data)
            if data == 0x01:
                dist, bias = lidar.get_distance()
                snow_depth = reference - dist
                lora.lora_send(snow_depth)
            elif data == 0x10:
                bias = lidar.get_bias_distance()
                reference = bias
                lora.lora_send(reference)
            elif data == 0x11:
                lora.lora_send(reference)
            elif data == 0x12:
                lora.lora_send(reference)
                reference = 0
            else:
                pass
        time.sleep(0.01)
```

Abbildung 28: main (code.py)

Zu Beginn werden alle zuvor beschriebenen Klassen instanziiert, danach wird der Wert der Referenz auf 0 gesetzt. Anschliessend folgt eine While schleife. Sofern daten ausgelesen wurden, wird die Luftfeuchtigkeit gemessen. Falls dieser Messwert den Standardwert von 65% nicht übersteigt, wird die entsprechende Funktion nach auswerten der erhaltenen Daten ausgeführt. Falls die Daten einen unbestimmten oder anderen als die gesuchten Werte entsprechen, passiert nichts.

6 Fehler

Die Software wurde mehrere Male getestet. Die einzelnen Softwareteile funktionieren, jedoch funktioniert die Lora Übertragung nicht korrekt. So läuft die Übertragung von Host zu Slave, jedoch nur stockend und unzuverlässig von Slave zu Host. Nach mehrehren Stunden an ausprobieren und verschiedenen Versuchen mit der Hardware habe ich mich dazu entschieden die Fehlersuche nicht mehr weiter fortzusetzten, da ich keine Ahnung mehr hatte an was es liegen kann, zudem das Projektstundenmaximal bereits übertreten ist.

Zudem gab es Fehler bei dem 3D Druck, jedoch sehe ich den Teil der Konstruktion und des Druckes nicht als direkten Teil des Projektes. Bei der Hardware gab es keine Fehler.

7 Projektzukunft

Das Projekt soll korrigiert werden, und mit einem Akku sowie einem Solarpanel ausgerüstet werden. Dann soll es ab nächstem Winter irgendwo in den Bergen aufgebaut und entsprechend in der Realität getestet werden.

8 Schlusswort

8.1 Fazit

Ich hatte zu Beginn grosse Mühe bei der Planung des Projektes, wodurch ich mich auch entschied, das Ganze zu überarbeiten. Nach der Überarbeitung ging es besser voran, da ich mir ein bessere Softwarekonzept überlegte, mit dem ich zufriedener war.
Im Großen und Ganzen hat mir das Projekt Spass bereitet, auch trotz Lieferverzögerungen, und anderen Rückschlägen bei der Programmierung. Der zeitrahmen konnte nicht eingehalten werden, da ich nicht nur zu wenige Zeit für die Programmierung sowie auch für das CAD-Zeichnen einplante.
Die grössten Probleme bei der Programmierung hatte ich beim Lidar und Lora Modul. Zwar gibt es eine Dokumentation zur Lidarlite garmin circuitpython Library, jedoch ist diese nicht besonders gut beschrieben. Ebenfalls sind gewisse Funktionen zur genauen Kalibrierung des Sensors nicht beschrieben. Lediglich in der C Library sind weitere Funktionen beschrieben. So musste ich teilweise stundenlange ausprobieren, bis ich endlich herausfinden konnte, ob ich nun direkt in die Register schreiben kann oder nicht, und ob der Sensor dies auch tut, was man ihm sagt.

Grundsätzlich bin ich mit dem Projekt nicht zufrieden, da die gewünschte Funktion so nicht erfüllt worden ist. Dennoch war es sehr lehrreich, nicht nur im Bereich der Programmierung, sondern auch im Bereich der Projektplanung.

9 Berechnungen

Es waren keine speziellen Berechnungen nötig.

10 Mitliegende Unterlagen

- Blatter_Mauro_schneemessapplikation_Arbeitsjournal.pdf
- Blatter_Mauro_schneemessapplikation_Zeitplan.pdf
- Blatter_Mauro_schneemessapplikation_Pflichtenheft.pdf
- Blatter_Mauro_schneemessapplikation_src

11 Abbildungsverzeichnis

Abbildung 1: Lora Übersicht 6
Abbildung 2: LIDAR Funktion 7
Abbildung 3: HW Übersicht 8
Abbildung 4: main.py 12
Abbildung 5: Objekt Flask 13
Abbildung 6: server run 14
Abbildung 7: request handler 15
Abbildung 8: Funktion get_timed_snow_depth 15
Abbildung 9: Funktion new_reference 16
Abbildung 10: Funktion get_reference 16
Abbildung 11: Funktion delete_reference 17
Abbildung 12: Private Funktion receive_data 17
Abbildung 13: LoraHandler 18
Abbildung 14: Funktion send_data 18
Abbildung 15: Funktion receive_data 19
Abbildung 16: Timer 20
Abbildung 17: Klasse bme680 21
Abbildung 18: Funktion get_temperature 21
Abbildung 19: Funktion get_altitude 22
Abbildung 20: Funktion get_humidity 22
Abbildung 21: Klasse LidarSensor 23
Abbildung 22: Funktion: get_distance 23
Abbildung 23: Funktion get_bias_distance 24
Abbildung 24: Unbenutzte Funktion set_configuration 24
Abbildung 25: Klasse Lora 25
Abbildung 26: Funktion lora_receive 25
Abbildung 27: Funktion lora_send 25
Abbildung 28: main (code.py) 26

12 Quellenverzeichnis

- https://rockpi.org/
- https://wiki.seeedstudio.com/XIAO-RP2040/
- https://oceanservice.noaa.gov/facts/lidar.html
- https://www.cambridge.org/core/journals/journal-of-glaciology/article/lidar-measurement-of-snow-depth-a-review/4419DF5C778946103080CB6187D434C0
- https://en.wikipedia.org/wiki/LoRa
- https://lora-alliance.org/
- https://learn.adafruit.com/welcome-to-circuitpython/what-is-circuitpython
- https://www.adafruit.com/product/4058
- https://learn.adafruit.com/adafruit-feather-32u4-radio-with-lora-radio-module/overview
- https://github.com/adafruit/Adafruit_CircuitPython_RFM9x